UNIVERSIDAD POPULAR DEL CESAR

Ingeniería de Sistemas e Informática

GUÍA DEL TALLER

Refactorización a Código Limpio

Sistema de Facturación — Restaurante El Buen Sabor

Programación de Computadores III · SS462 · Cuarto Semestre

Docente: Ing. Alfredo David Bautista Romero

1. ¿Qué van a encontrar?

El proyecto RestauranteBuenSabor es un sistema de facturación para un restaurante que funciona correctamente, pero está escrito con el máximo de malas prácticas posibles. Su misión es refactorizarlo hasta convertirlo en código limpio, sin alterar el comportamiento del programa.

> ⚠ *El programa produce resultados correctos. El problema no es que falle — el problema es que es imposible de mantener, entender y modificar. Eso es exactamente lo que pasa en la industria real.*

1.1 Estructura de archivos entregada

```
RestauranteBuenSabor/
├── src/
│   └── restaurante/
│       ├── Main.java        — punto de entrada y menú interactivo
│       ├── Datos.java       — almacén global de datos del sistema
│       ├── Proceso.java     — lógica de cálculo de la factura
│       ├── Utilidades.java  — métodos auxiliares
│       └── Imprimir.java    — todo lo que se imprime en pantalla
└── README.md                — instrucciones para compilar y ejecutar
```

2. ¿Qué hace el sistema?

El programa simula la operación de caja de un restaurante. Al ejecutarlo aparece un menú con 6 opciones que se repite en bucle hasta que el usuario elige salir:

```
========================================
    RESTAURANTE EL BUEN SABOR
    Calle 15 #8-32, Valledupar
    NIT: 900.123.456-7
========================================
1. Ver carta
2. Agregar producto al pedido
3. Ver pedido actual
4. Generar factura
5. Nueva mesa
0. Salir
========================================
Seleccione una opcion:
```

2.1 Carta del restaurante (Opción 1)

#	Producto	Precio
1	Bandeja Paisa	$32.000
2	Sancocho de Gallina	$28.000
3	Arepa con Huevo	$8.000

4	Jugo Natural	$7.000
5	Gaseosa	$4.500
6	Cerveza Poker	$6.000
7	Agua Panela	$3.500
8	Arroz con Pollo	$25.000

2.2 Reglas de negocio (Opción 4 — Generar factura)

- Subtotal: suma de precio × cantidad para cada producto pedido.
- Descuento 5%: se aplica si hay más de 3 productos diferentes en el pedido.
- IVA 19%: se calcula sobre el subtotal después del descuento.
- Propina 10%: se aplica si el subtotal (antes del IVA) supera $50.000.
- Total: subtotal con descuento + IVA + propina.

Ejemplo de factura generada (Bandeja Paisa x2 + Jugo Natural x2):

```
FACTURA No. 001
----------------------------------------
Bandeja Paisa        x2      $64.000
Jugo Natural         x2      $14.000
----------------------------------------
Subtotal:                   $78.000
IVA (19%):                  $14.820
Propina (10%):              $ 9.282
----------------------------------------
TOTAL:                      $102.102
```

3. Catálogo de Malas Prácticas

A continuación, se documenta cada mala práctica presente en cada archivo, con su ubicación y descripción. Usen esto como lista de verificación durante la refactorización.

3.1 Datos.java — El God Object

Almacena todos los datos del sistema en variables estáticas públicas. Cualquier clase puede leer y modificar cualquier dato directamente, sin control ni encapsulamiento.

#	Problema	Dónde	Descripción
1	Arrays paralelos		
2	Todo public static	Clase Datos línea 12 hasta la 18. Clase Proceso linea 12 y línea 47. Clase Imprimir línea 12, 22, 34 y 79. Clase Utilidades línea 12, 23 y 31.	La clase Datos con atributos nom, p, cant, ms, tot, x, tmp públicos. La clase Proceso con los atributos hacerTodo Y procesar. La clase Imprimir con los atributos mostrarCarta, mostrarPedido, imprimirFacturaCompleta y imprimirFacturaResumen. La clase Utilidades con los atributos calcular, validar y reiniciar.

3	Nombre hardcodeado		
4	Variable tmp sin propósito		
5	Sin constructor ni métodos	Ninguna de las clases	Ninguna de las 5 clases tiene constructor explícito ni métodos de instancia todas son static
6	Nombres de 1-3 letras	Clase Datos línea 12, 13, 15, 16, 17 y 18. Clase Proceso línea 13, 47 y 48. Clase imprimir línea 23, 35 y 80. Clase Utilidades línea 12. Clase RestauranteElBuenSabor línea 16, 40, 42, 93 y 123.	La clase datos presenta atributos nom, p, ms, est, tot, nf, x y tmp. La clase proceso presenta variables sub, tot, aux, a, b, c, d, e, f, g, res y tmp. La clase Imprimir presenta una variable sub, tot, aux, sep. La clase utilidades presenta parámetros pr, cn, dc, iv, pp, ni, ap, res, tmp. La clase RestauranteElBuenSabor presenta valiables op, x, aux, tmp, m, sc, n, c, r, sc2

3.2 Proceso.java — El método que hace todo

Contiene la lógica de cálculo del total de la factura en un único método de más de 80 líneas.

#	Problema	Dónde	Descripción
7	hacerTodo() con múltiples responsabilidades	Clase Proceso línea 12 hasta la 46.	Realiza 5 cosas distintas calcula subtotal, aplica descuentos, calcula iva, aplica propina y modifica el estado global.
8	IVA duplicado 3 veces		
9	Magic numbers sin nombre		
10	Condicionales anidados 4 niveles		
11	Código comentado 'versión anterior'	Clase Proceso línea 32	Era una forma más simple de calcular el total.
12	procesar() con 7 parámetros		
13	Variable aux innecesaria	Clase Utilidades línea 13, 21,22. Clase RestauranteElBuenSabor línea 16, 51, 56, 97, 113. Clase Proceso línea 48, 55, 56.	En la clase Utilidades la variable aux2 solo copia el valor de res para retornarlo se puede simplificar a return res. En la clase RestauranteElBuenSabor la variable aux se asigna en varios puntos, pero nunca se usa para nada. En la clase Proceso la variable tmp solo copia el valor de iva y luego se suma a res se puede reemplazar directamente por res=res+iva.
14	Comentarios que repiten el código	Clase Proceso línea 16, 25, 29, 43, 52 y 54. Clase imprimir línea 27, 29, 36 y 59.	En todas las clases los comentarios sobran ya que no son necesarios.

		La clase RestauranteElBuenSabor línea 33, 37, 59, 66, 69, 80, 90 y 117. Clase Utilidades línea 14 y 19.	
15	Comentarios de cierre inútiles	Clase proceso línea 19, 31, 44. Clase imprimir línea 20, 31, 40, 50, 63, 68, 85. Clase RestauranteElBuenSabor línea 57, 58, 71, 77, 87, 107, 129, 130, 131, 132. Clase utilidades línea 27	Los comentarios tipo fin while, fin if, fin main son innecesarios porque el IDE ya resalta las llaves correspondientes haciendo clic sobre ella.

3.3 Utilidades.java — Nombres engañosos

Contiene métodos auxiliares: calcular, validar y reiniciar el pedido.

#	Problema	Dónde	Descripción
16	calcular() con 7 parámetros	Clase utilidades línea 12. Clase proceso línea 47.	Se recomienda máximo **3** parámetros y se están usando 7 lo que puede ocasionar problemas.
17	validar() con efecto secundario oculto	Clase Utilidades línea 23 hasta la 30.	El método validar en la línea 29 si no hay nada reinicia modifica el estado global es decir resetea.
18	Nombre del restaurante hardcodeado (3ra vez)		
19	Bloque de ~20 líneas de código comentado	Clase utilidades línea 32 hasta la 47.	Cuenta con 16 líneas de comentario lo que es excesivamente innecesario.
20	Variables tmp y aux2 redundantes	Clase utilidades línea 13, 21 hasta la 22. Clase proceso línea 48, 55, 56.	Clase utilidades aux2 en calcular()solo existe para copiar el valor de res y retornarlo. tmp en calcular() solo guarda el resultado de res*iv para sumarlo a res en la misma línea. Clase proceso tmp copia el valor de iva y luego se suma a res pero ya existe iva con el mismo valor.

3.4 Imprimir.java — Código copiado y pegado

Contiene todos los métodos de impresión en consola.

#	Problema	Dónde	Descripción
21	imprimirFacturaCompleta() con 3 responsabilidades	Clase imprimir línea 34 hasta 78	Tiene la responsabilidad de calcular totales, imprimir factura y actualizar el estado global.
22	Cálculo del total duplicado	Clase proceso línea 12 hasta la 46.	Clase proceso en hacer todo() esta la primera copia.

		Clase imprimir línea 35 hasta la 50 y de 80 a 95.	Clase imprimir en imprimirFacturaCompleta() y imprimirFacturaResumen() se encentra las otras dos.
23	80% del código idéntico entre métodos	Clase imprimir línea 34 hasta la 78 y de la 79 hasta 110.	Los métodos imprimirFacturaCompleta() y imprimirFacturaResumen() tiene aproximadamente el 80 de similitud.
24	Nombre del restaurante hardcodeado (4ta vez)		
25	Formato printf repetido sin constante		

3.5 Main.java — El caos central

Punto de entrada del programa. El método main mezcla presentación, validación y lógica de negocio en más de 100 líneas.

#	Problema	Dónde	Descripción
26	Método main de 100+ líneas	Clase RestauranteElBuenSabor Línea 14 hasta la 132.	Debería tener máximo de 20 a 30 líneas y hace de todo.
27	Dos Scanner para el mismo System.in	Clase RestauranteElBuenSabor línea 123 y 129.	Se crea cada vez que el usuario ingresa una opción inválida.
28	Variables sin uso		
29	Lógica de negocio en el main	Clase RestauranteElBuenSabor línea 45 hasta la 57, 59 hasta 63, 42 hasta 67, 120 hasta 129 y 106, 107.	Tiene 5 lógicas de negocio las cuales son gestión de mesa, agregar producto al pedido, validación de producto y cantidad, control de intentos inválidos y reinicio de variables tras factura fallida
30	Nombre del restaurante hardcodeado (5ta vez)		

3.6 Problemas de formato visual (todos los archivos)

#	Problema	Dónde	Descripción
31	Sin indentación	Todas las clases	Ninguna clase presenta identacion donde corresponde.
32	Llaves inconsistentes	Clase proceso línea 18, 19, 20, 21, 27, 28, 31, 36, 41, 44, 53. Clase imprimir línea 41, 42, 46, 47, 50, 62, 68, 86. Clase restauranteelbuensabor	En todas las clase tiene llaves de cierre y apertura pegado al código en la misma línea lo que puede dificultar el entendimiento del código.

		32, 36, 53, 55, 56, 64, 67, 69,71, 77, 118, 129. Clase utilidades línea 16, 18, 22, 26, 29, 48.	
33	Llaves de cierre apiladas		
34	Múltiples statements por línea		

Total: 34 malas prácticas identificadas y documentadas en el proyecto.

Referencia principal: Robert C. Martin — Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2009.

Fuente complementaria: Shahan Chowdhury — The Clean Code Handbook. FreeCodeCamp, Enero 2025.